ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ

МОСКОВСКИЙ ПОЛИТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ

Факультет Информационных технологий
Кафедра Информатики и информационных технологий

направление подготовки
09.03.02 «Информационные системы и технологии»

ПРАКТИЧЕСКАЯ РАБОТА № 2

Дисциплина: Backend-разработка

Тема: HTTP методы и передача данных на сервер

Выполнил(а): студент(ка) группы 241-331

Давлетова Аделина Сергеевна
(Фамилия И.О.)

Дата, подпись _____ _____
(Дата) (Подпись)

Проверил: _____ _____
(Фамилия И.О., степень, звание) (Оценка)

Дата, подпись _____ _____
(Дата) (Подпись)

Замечания: _____

_____

_____

Москва
2026

Цель занятия:

Изучить способы передачи данных на сервер в ASP.NET Core Web API.

Задание:

1. Создать проект ASP.NET Core Web API.
2. Реализовать обработчик app.MapGet(...) с передачей данных через query-параметры.
3. Реализовать обработчик app.MapPost(...) с приёмом данных через тело запроса.
4. Реализовать обработчик app.MapPost(...) с приёмом данных и через query-параметры и через тело запроса.
5. Реализовать обработчик app.MapPut(...) с приёмом данных из тела запроса.
6. Реализовать обработчик app.MapPatch(...) для частичного обновления. данных.
7. Проверить работу всех обработчиков через браузер или HTTP-клиент.
8. Сравнить назначение и особенности каждого типа HTTP-запроса.

Ход работы:

В ходе работы было создано приложение ASP.NET Core с использованием шаблона Web API для изучения способов передачи данных на сервер.

После создания проекта был настроен минимальный API и подключён Swagger для тестирования HTTP-запросов, фрагмент кода Program.cs:

```
var builder = WebApplication.CreateBuilder(args);

builder.Services.AddEndpointsApiExplorer();
builder.Services.AddSwaggerGen();

var app = builder.Build();

if (app.Environment.IsDevelopment())
{
```

```
    app.UseSwagger();
    app.UseSwaggerUI();
}
```

Проверка работы обработчиков выполнялась с помощью Swagger UI, который позволяет отправлять HTTP-запросы и просматривать ответы сервера.

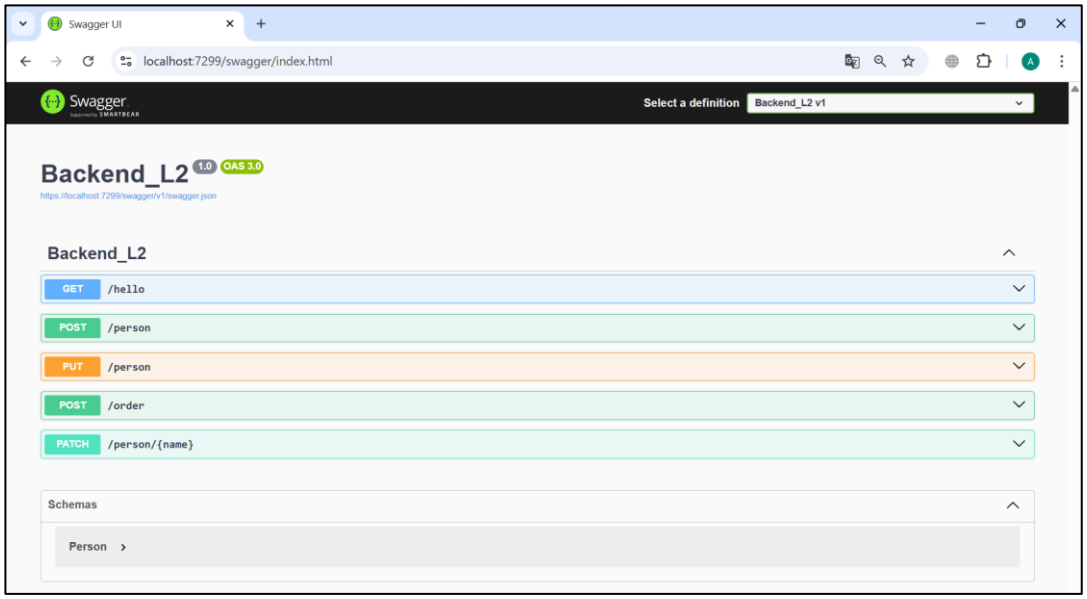

Рисунок 1, Интерфейс Swagger UI

Реализация GET-запроса с query-параметрами

Был реализован обработчик MapGet, принимающий параметры через строку запроса URL:

```
app.MapGet("/hello", (string name, int age) =>
{
    return $"Привет, {name}! Тебе {age} лет.";
    });
```

Пример запроса:

/hello?name=Adelina&age=21

Сервер возвращает текстовый ответ с переданными данными.

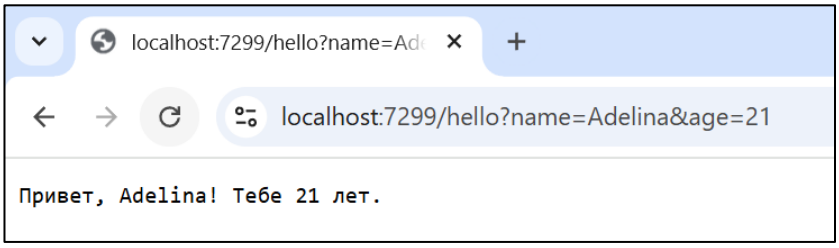

Рисунок 2. Результат выполнения GET-запроса в браузере

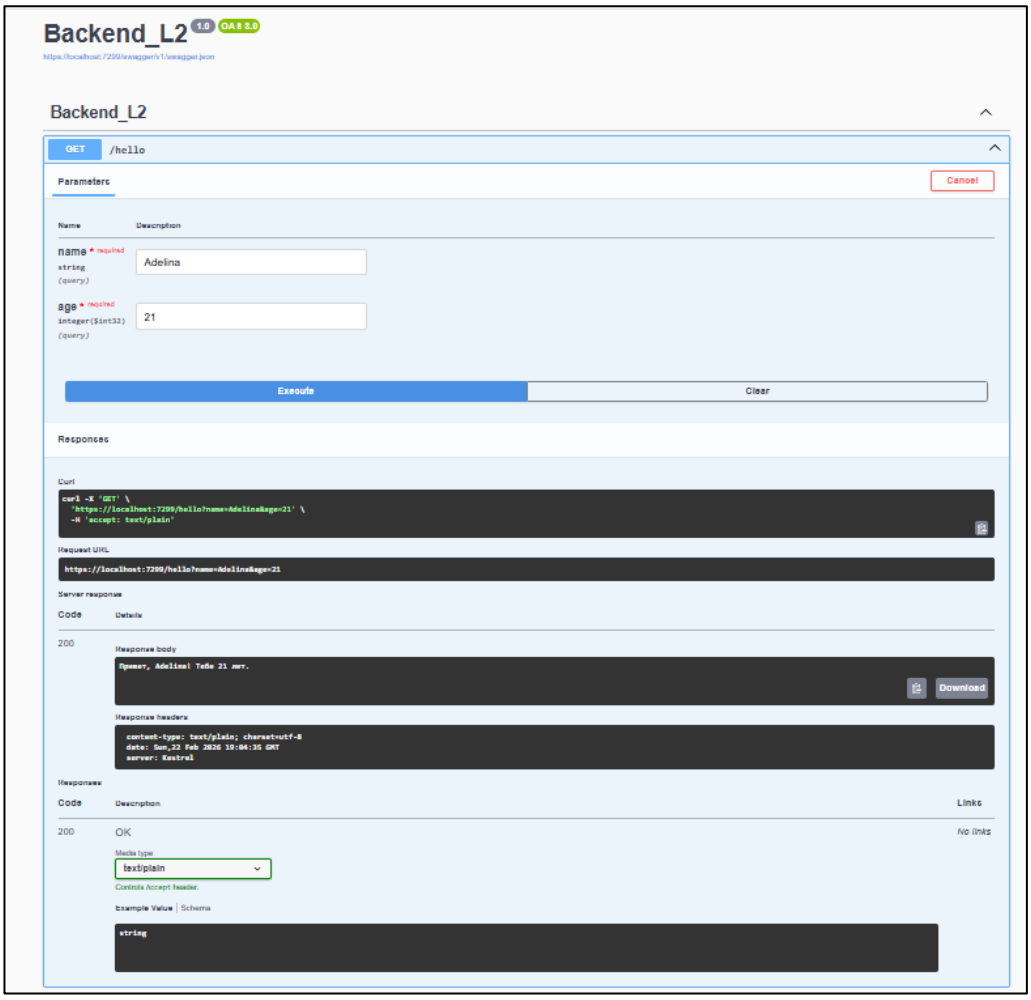

Рисунок 3. Проверка GET-запроса через Swagger

Реализация POST-запроса с передачей данных в теле запроса

Была создана модель Person для приёма JSON-данных.

Листинг кода Person.cs:

```
public class Person
{
    public string Name { get; set; }
    public int Age { get; set; }
}
```

POST-запрос принимает объект пользователя и возвращает сообщение с полученными данными:

```
app.MapPost("/person", (Person person) =>
{
    return $"Получен человек: {person.Name}, возраст {person.Age}";
        });
```

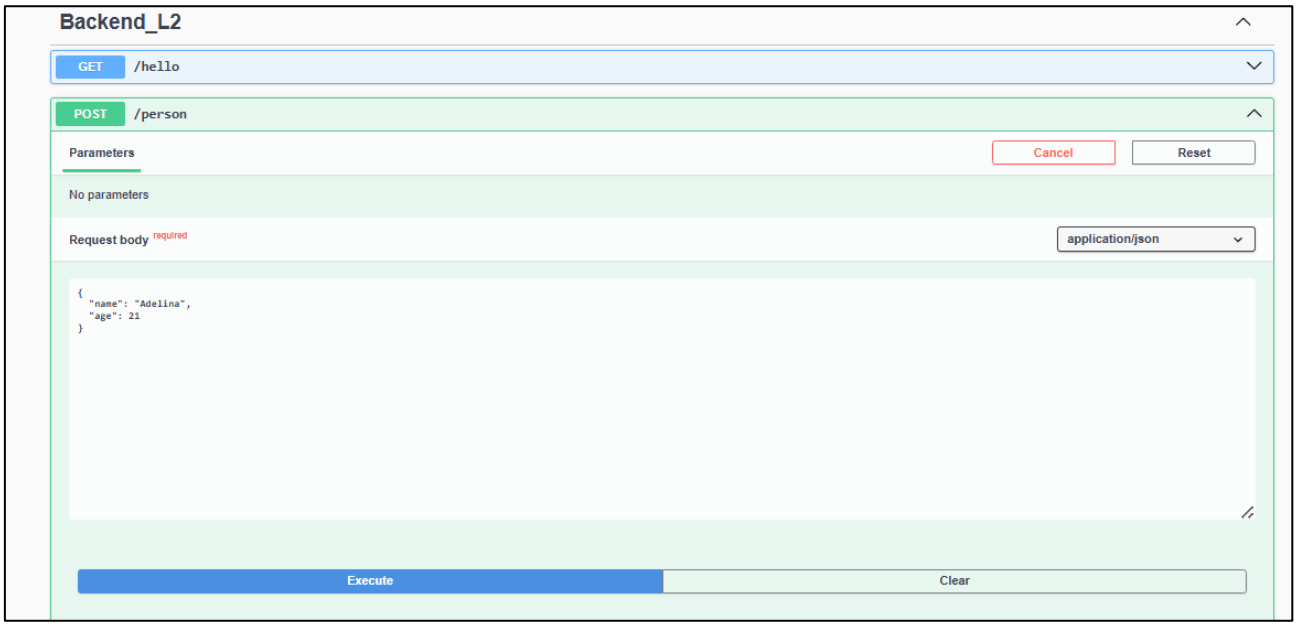

Рисунок 4. Отправка JSON через Swagger

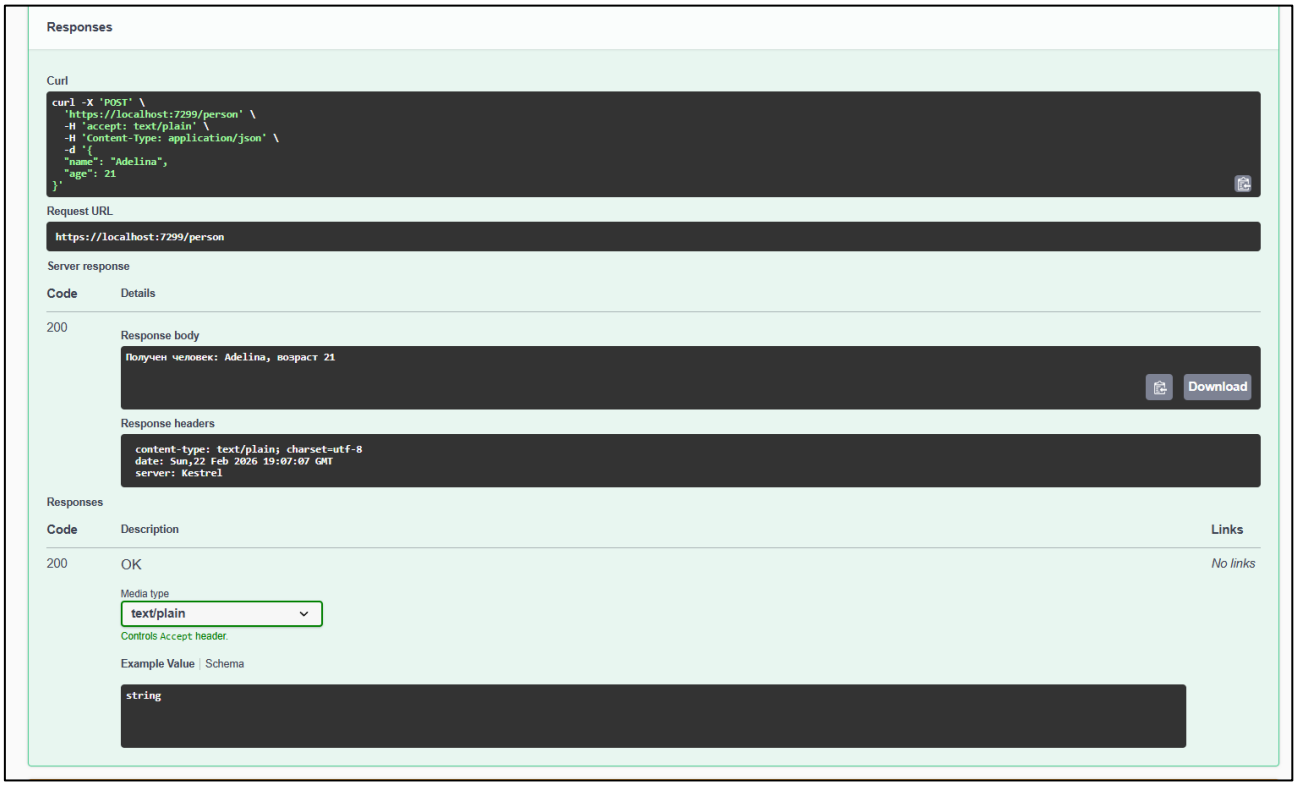

Рисунок 5. Ответ на POST-запрос

Реализация POST-запроса с передачей данных через query и тело запроса

Был создан обработчик, принимающий:

- идентификатор через query-параметр
- объект пользователя через тело запроса

Листинг Program.cs с реализацией данного POST-запроса:

```
app.MapPost("/order", (int id, Person person) =>
{
    return $"Заказ №{id} оформлен на {person.Name}";
});
```

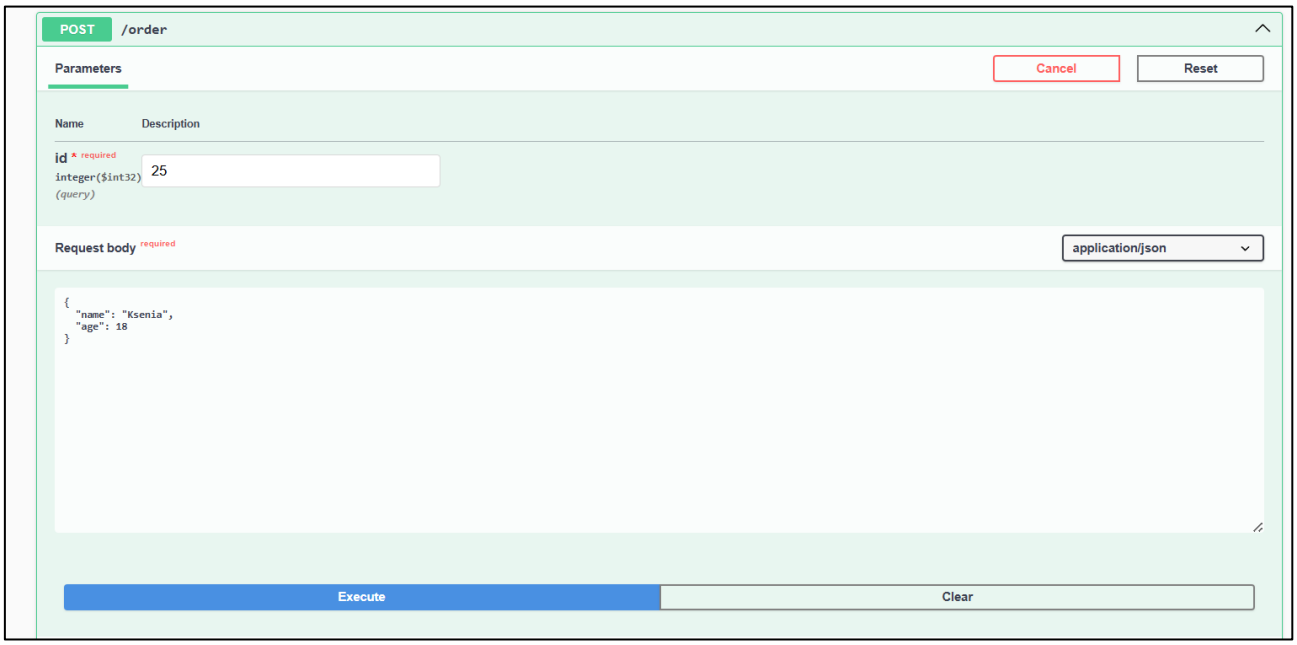

Рисунок 6. Передача параметра id через query и объекта Person через тело запроса

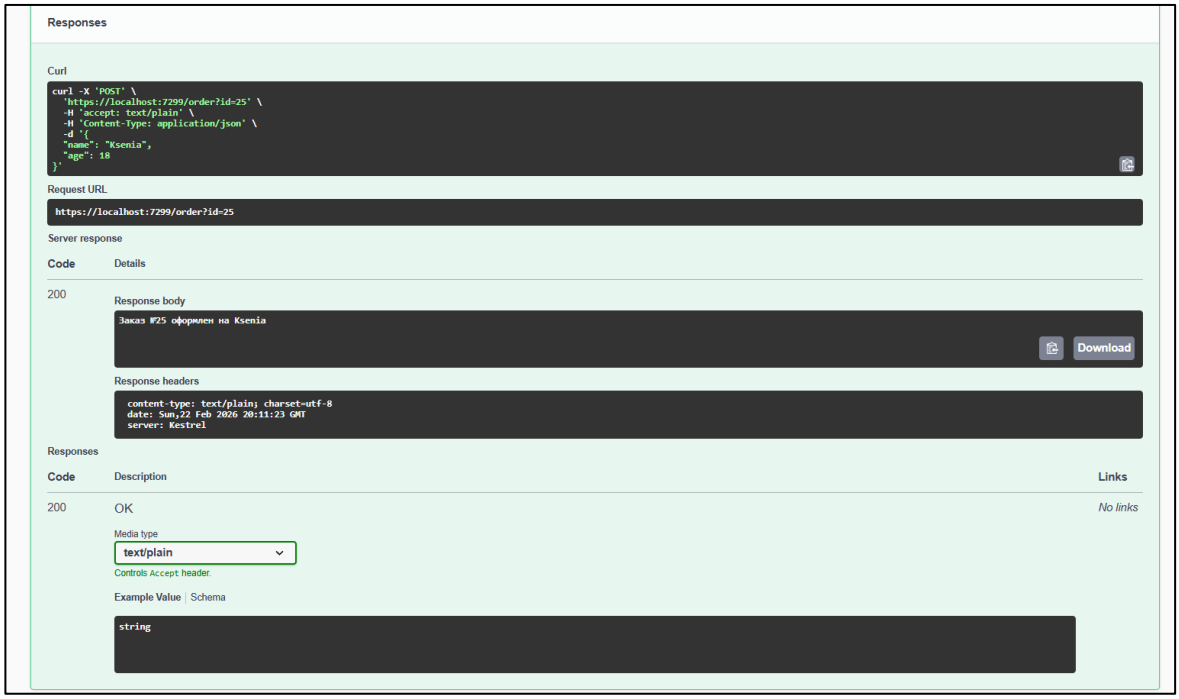

Рисунок 7. Выполнение POST-запроса с передачей данных через URL и тело запроса

Реализация PUT-запроса для обновления данных

PUT-запрос используется для полного обновления объекта. Сервер принимает объект и возвращает обновлённые данные:

```
app.MapPut("/person", (Person person) =>
{
    return $"Данные обновлены: {person.Name}, {person.Age}";
});
```

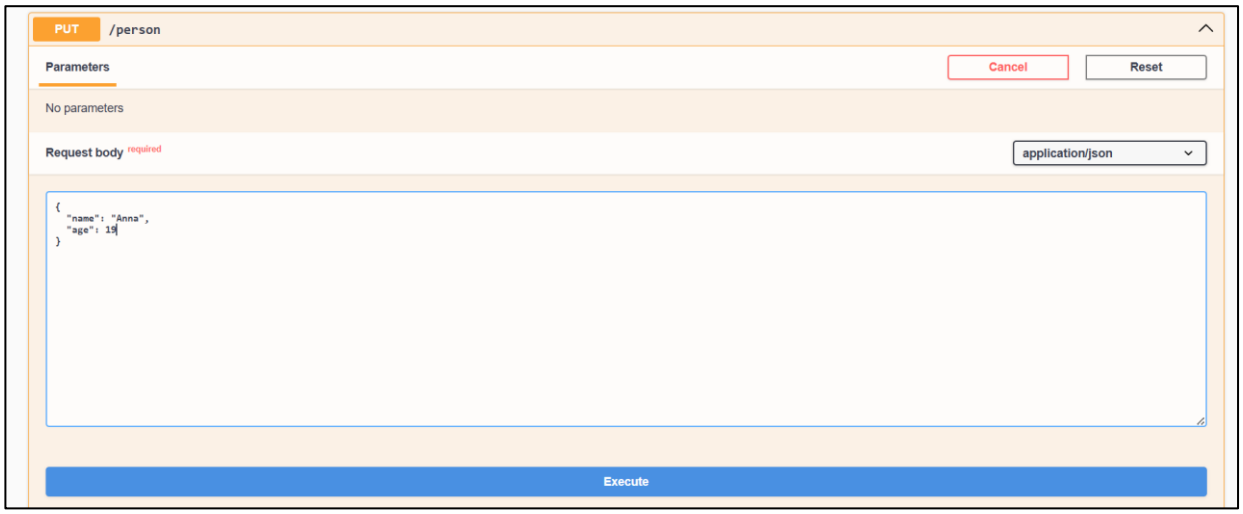

Рисунок 8. Отправка JSON через Swagger

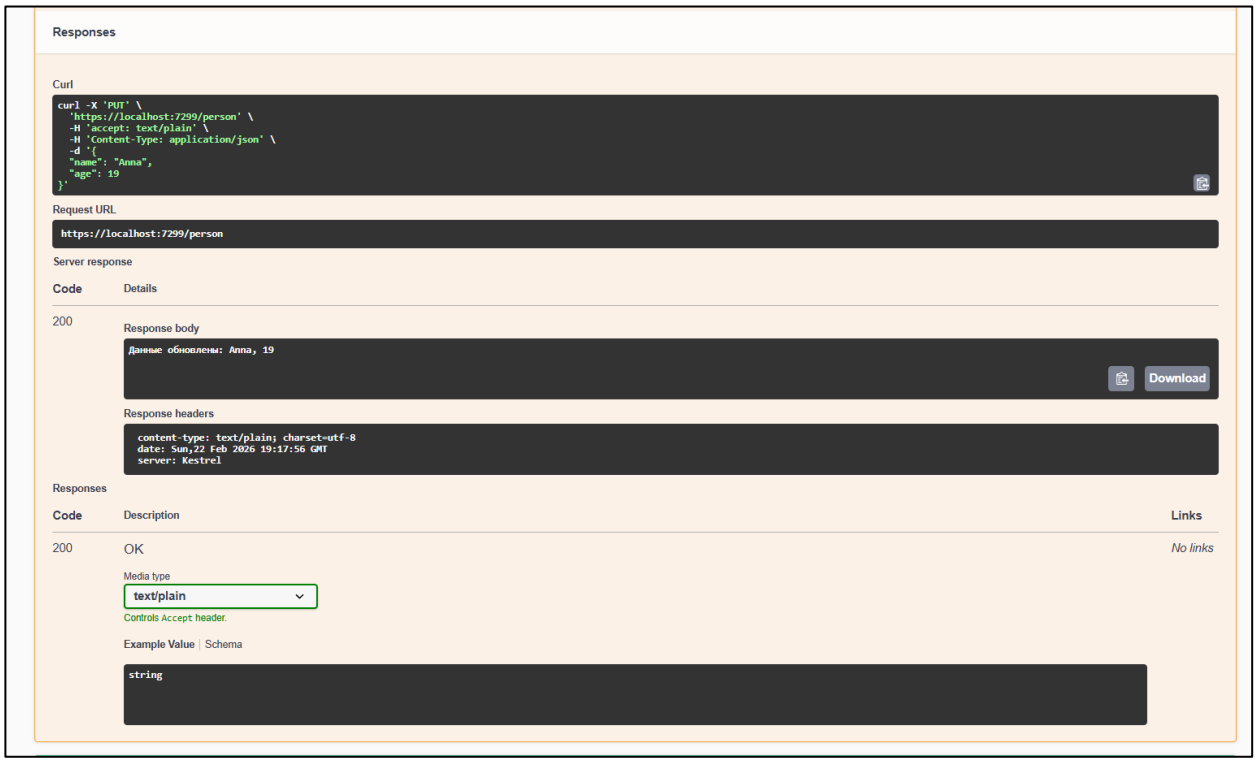

Рисунок 9. Проверка PUT-запроса

Реализация PATCH-запроса для частичного обновления данных

PATCH-запрос реализован для изменения отдельных полей объекта без полной замены:

```
app.MapPatch("/person/{name}", (string name, int age) =>
{
   return $"Возраст пользователя {name} обновлён на {age}";
        });
```

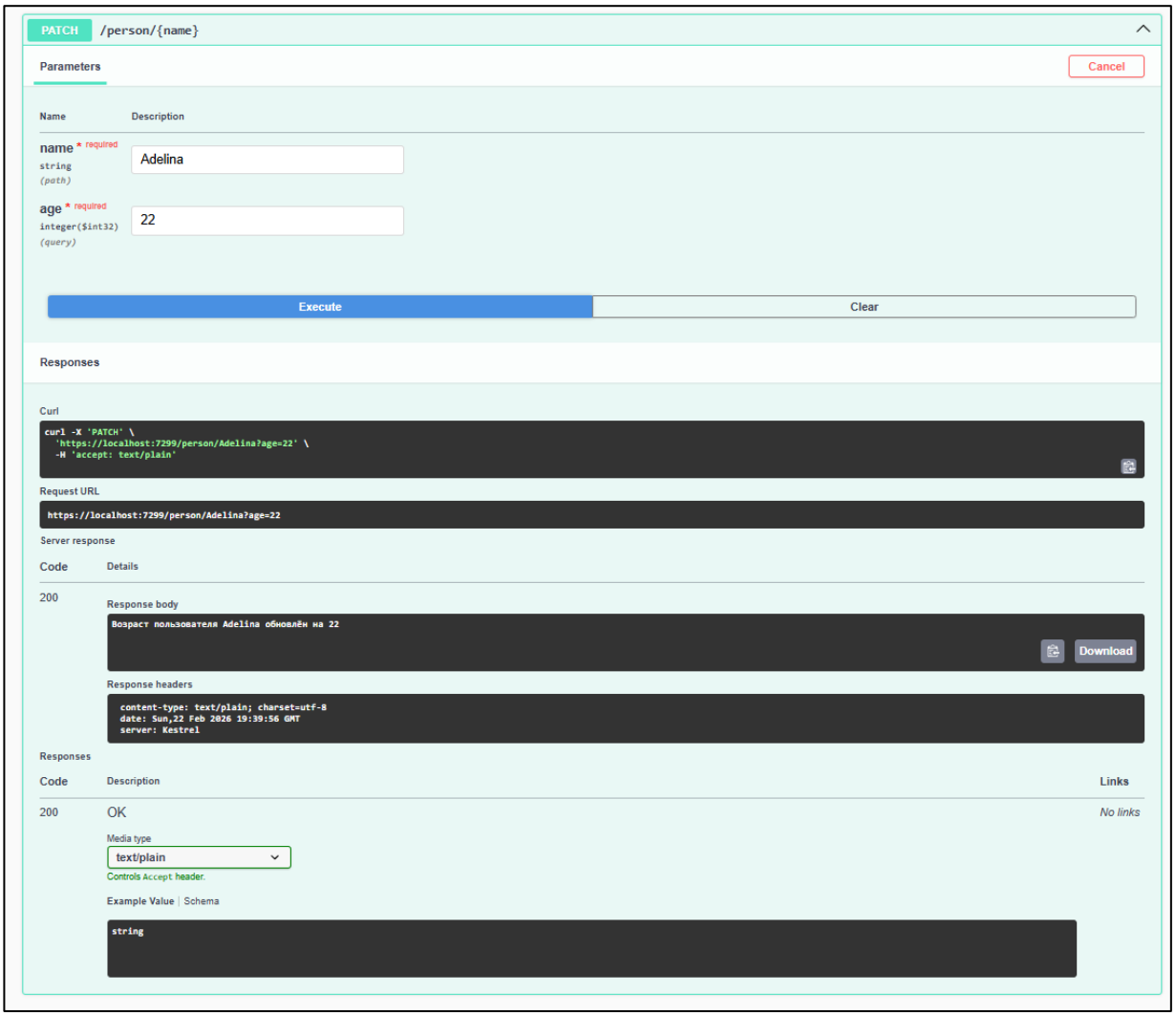

Рисунок 10. Результат частичного обновления

В ходе выполнения работы были изучены особенности HTTP-методов:

– GET — получение данных, параметры передаются в URL.

– POST — отправка и создание данных, передача через тело запроса.

– PUT — полное обновление объекта.

– PATCH — частичное изменение данных.

Ссылка на репозиторий: https://github.com/aeidln/Backend.git

Вывод:

В ходе лабораторной работы были изучены способы передачи данных на сервер в ASP.NET Core Web API. Были реализованы обработчики GET, POST, PUT и PATCH, а также рассмотрены различные способы передачи данных через query-параметры и тело запроса. Получен практический опыт тестирования API с использованием Swagger. Работа позволила понять назначение различных HTTP-методов и их применение при разработке веб-сервисов.